

课程笔记

扩散语言模型：从 MDLM 到 SoLM

基于公开课程资料整理

2025 年秋季

CS492C: Diffusion and Flow Models

Simple and Effective Masked Diffusion Language Models

GUEST LECTURE 1
SUBHAM SAHOO

Fall 2025
KAIST

CS492(C): Diffusion and Flow Models (Fall 2025)

视频作者/频道: Subham Sahoo (Guest Lecture)

发布日期: 2025

视频时长: 40 分钟

视频链接: <https://www.youtube.com/watch?v=eA1EnwoMxf0>

目录

1 引言：为什么需要扩散语言模型	2
1.1 扩散语言模型发展时间线	2
1.2 本章小结	2
2 MDLM：掩码扩散语言模型	3
2.1 设计目标与核心思想	3
2.2 前向过程：掩码噪声	3
2.3 反向后验：去掩码函数	4
2.4 生成过程 p_θ	4
2.5 本章小结	4
3 训练 MDLM	5
3.1 ELBO 损失函数	5
3.2 AR 训练与 MDLM 训练的对比	5
3.3 采样过程详解	5
3.4 本章小结	6
4 实验结果与应用	7
4.1 语言建模性能	7
4.1.1 LM1B 数据集	7
4.1.2 零样本迁移能力	7
4.2 大规模验证：LaDa (8B MDLM)	7
4.3 超越语言：离散数据建模	7
4.4 可控生成	8
4.5 本章小结	8
5 MDLM 的局限性：KV 缓存问题	9
5.1 KV 缓存回顾	9
5.2 为什么 MDLM 不支持 KV 缓存	9
5.3 本章小结	9
6 SoLM：融合 AR 与扩散的排序语言模型	10
6.1 核心思路：排序 + 因果注意力	10
6.2 推理过程与 KV 缓存	10
6.3 AR 与扩散的插值	11
6.4 本章小结	11
7 方法对比与性能分析	12
7.1 MDLM vs BD3LM vs SoLM	12
7.2 速度-质量权衡曲线	12
7.3 本章小结	12

8 总结与延伸	13
8.1 核心要点回顾	13
8.2 工业应用现状	13
8.3 拓展阅读	13

1 引言：为什么需要扩散语言模型

自回归(Autoregressive, AR)模型长期以来是语言生成领域的主流范式。从 GPT 系列到 LLaMA, 这些模型以严格的从左到右(left-to-right)方式逐词生成文本, 已经取得了巨大的成功。然而, AR 模型存在一个根本性的速度瓶颈: 生成 N 个 token 需要 N 步前向传播, 无法实现真正的并行生成。

扩散语言模型的核心优势

扩散语言模型(Diffusion Language Models)摆脱了从左到右的约束, 能够以任意顺序生成文本, 最重要的是可以**并行生成多个词**。这使得它们在推理速度上具有显著的优势。

2025 年以来, 扩散语言模型引起了工业界的广泛关注。Google 发布了 Gemini Diffusion, ByteDance 推出了 Seed Diffusion, Inception Labs 发布了 Mercury Coder。这些模型不仅性能出色, 而且在推理速度上远超传统 AR 模型。Forbes 和 Fortune 等主流商业媒体也开始报道这一技术变革。

本次客座讲座的主讲人 Subham Sahoo 是 Cornell 大学博士毕业生, 其博士研究聚焦于扩散语言模型。他是 MDLM (Masked Diffusion Language Models) 和 SoLM (Sorting-based Language Models) 等重要工作的第一作者。本讲座将深入讲解这两篇论文的核心思想与技术细节。

1.1 扩散语言模型发展时间线

从 2022 到 2025: 关键里程碑

1. **2022 年**: 扩散语言模型首次被提出。这些模型能以随机顺序生成词, 但文本质量远落后于 AR 模型, 社区反应冷淡。
2. **2024 年初**: Lou et al. 证明扩散 LM 搭配现代 Transformer 架构可以匹配 GPT-2 级别的 AR 模型质量, 同时显著更快。
3. **2024 年中**: MDLM 发布, 重新推导并简化了离散扩散训练算法, 在小规模文本数据集上匹配 AR 模型性能。
4. **2025 年初**: Diffusion Duality 论文提出蒸馏算法, 使模型能用仅 10 步生成 1000 词的序列, 质量几乎无损。
5. **2025 年**: SoLM 发布, 融合 AR 与离散扩散, 支持 KV 缓存, 推理速度实现数量级提升。

1.2 本章小结

扩散语言模型代表了文本生成范式的一次重要转变。从 2022 年的概念提出到 2025 年的工业化部署, 这一领域经历了快速发展。MDLM 框架是当前多个工业级扩散语言模型(如 ByteDance 的 Seed Diffusion)的基础。

2 MDLM: 掩码扩散语言模型

2.1 设计目标与核心思想

MDLM (Masked Diffusion Language Model) 的目标是设计一个支持**并行采样**的语言模型。与 AR 模型逐词生成不同, MDLM 从一个全部被掩码 (mask) 的序列出发, 逐步恢复出完整的文本。生成过程的直觉:

1. 初始状态: 一个长度为 L 的全 mask 序列 $[M, M, \dots, M]$
2. 将序列送入一个 BERT-like 双向 Transformer
3. 模型预测所有 mask 位置的词
4. 选择性地揭示部分预测, 其余重新 mask
5. 重复步骤 2-4, 直到所有位置都被揭示

MDLM 与 AR 的本质区别

- **AR 模型**: 使用**因果注意力** (Causal Attention), 每个 token 只依赖左侧上下文, 逐个生成
- **MDLM**: 使用**双向注意力** (Bidirectional Attention), 每个 token 依赖整个上下文, 可同时生成多个词

2.2 前向过程: 掩码噪声

扩散模型的核心框架是: 定义一个已知的前向**腐蚀过程** q , 然后学习其**逆过程** p_θ 。
在 MDLM 中, 前向过程就是**掩码操作**。给定干净文本 $\mathbf{x}$, 通过逐步遮盖词来产生噪声序列。

信号水平

MDLM 使用信号水平 $\alpha_t \in [0, 1]$ 来控制噪声程度:

- $\alpha_t = 1$: 完全干净的句子 $\mathbf{x}$ (时间 $t = 0$)
- $\alpha_t = 0$: 完全被 mask 的序列 (时间 $t = 1$)

从干净句子到中间噪声状态 $\mathbf{z}_t$ 的过程非常简单: 对每个干净词抛一枚硬币, 以概率 $1 - \alpha_t$ 将其替换为 mask token。

形式化地, 前向过程可以写为:

$$q(\mathbf{z}_t | \mathbf{x}) = \prod_{i=1}^L q(z_t^{(i)} | x^{(i)})$$

其中每个位置独立地进行掩码:

$$q(z_t^{(i)} | x^{(i)}) = \begin{cases} \alpha_t & \text{if } z_t^{(i)} = x^{(i)} (\text{保留原词}) \\ 1 - \alpha_t & \text{if } z_t^{(i)} = M (\text{替换为 mask}) \end{cases}$$

- $\mathbf{x}$: 干净文本序列
- $\mathbf{z}_t$: 时间 t 对应的噪声序列
- α_t : 时间 t 的信号水平
- L : 序列长度
- M : 特殊的 mask token

2.3 反向后验：去掩码函数

反向后验

这是整个讲座中最重要的公式之一。反向后验 $q(\mathbf{z}_s | \mathbf{z}_t, \mathbf{x})$ 描述了：给定噪声序列 $\mathbf{z}_t$ 和干净文本 $\mathbf{x}$ ，如何从更嘈杂的状态 $\mathbf{z}_t$ 过渡到更干净的状态 $\mathbf{z}_s$ （其中 $s < t$ ）。

对于每个位置 i ，反向后验的行为取决于该位置当前的状态：

- **已揭示的 token** ($z_t^{(i)} \neq M$): 保持不变，即 $z_s^{(i)} = z_t^{(i)}$
- **mask token** ($z_t^{(i)} = M$): 以概率 $\frac{\alpha_s - \alpha_t}{1 - \alpha_t}$ 被去掩码为干净词 $x^{(i)}$ ，否则保持 mask

反向后验本身并不能直接用于生成

注意 $q(\mathbf{z}_s | \mathbf{z}_t, \mathbf{x})$ 需要知道干净文本 $\mathbf{x}$ ，但在生成时我们恰恰不知道 $\mathbf{x}$ 是什么。因此，挑战在于学习一个**去噪网络** $x_\theta(\mathbf{z}_t, t)$ 来估计 $\mathbf{x}$ ，然后将估计值代入反向后验公式得到 $p_\theta(\mathbf{z}_s | \mathbf{z}_t)$ 。

2.4 生成过程 p_θ

在推理时，我们使用训练好的去噪 Transformer x_θ 来近似反向过程：

1. 将噪声序列 $\mathbf{z}_t$ 送入双向 Transformer
2. 模型对每个 mask 位置输出一个词表上的**类别分布**
3. 从分布中采样，填充所有 mask 位置
4. 选择性地重新 mask 部分预测（根据时间步进决定保留多少）
5. 重复上述过程，直到所有位置都被揭示

Copy-over 操作

MDLM 的 Transformer 内含一个 copy-over 机制：对于已经是干净（非 mask）的位置，模型直接复制输入作为输出，不做修改。这意味着交叉熵损失实际上**只在 mask 位置上计算**，大幅简化了训练。

2.5 本章小结

MDLM 的核心设计可以概括为：（1）前向过程通过逐步掩码将文本转化为全 mask 序列；（2）学习一个双向 Transformer 来估计干净文本；（3）利用反向后验公式逐步去掩码以生成文本。

3 训练 MDLM

3.1 ELBO 损失函数

在 AR 模型中，训练目标是直接最大化对数似然 $\log p(\mathbf{x})$ 。在扩散模型中无法精确计算似然，因此使用**证据下界**（Evidence Lower Bound, ELBO）作为训练目标。

MDLM 的核心贡献之一就是推导出了一个**极其简洁**的连续时间 ELBO：

$$\mathcal{L}_{\text{MDLM}} = \mathbb{E}_{t \sim \mathcal{U}(0,1), \mathbf{z}_t \sim q(\mathbf{z}_t|\mathbf{x})} \left[\frac{\alpha'_t}{1 - \alpha_t} \sum_{i: z_t^{(i)} = \text{M}} -\log x_\theta^{(i)}(\mathbf{z}_t, t) \right]$$

各项含义：

- $\mathbb{E}_{t \sim \mathcal{U}(0,1)}$ ：从 $[0, 1]$ 均匀采样时间步 t
- $\mathbf{z}_t \sim q(\mathbf{z}_t|\mathbf{x})$ ：对干净文本 $\mathbf{x}$ 施加噪声得到 $\mathbf{z}_t$
- $\frac{\alpha'_t}{1 - \alpha_t}$ ：权重项，即去掩码的概率（chance of unmasking）
- $-\log x_\theta^{(i)}(\mathbf{z}_t, t)$ ：交叉熵损失，仅在 mask 位置上计算

连续时间极限

MDLM 论文的关键洞察：当离散化步数 $T \rightarrow \infty$ 时，离散时间 Markov 链收敛到连续时间 Markov 链，从而得到上述简洁的损失函数。这一简化极大地方便了训练和评估。

3.2 AR 训练与 MDLM 训练的对比

AR 模型和 MDLM 的训练流程非常相似，仅有三处关键差异：

表 1: AR 模型与 MDLM 训练对比

方面	AR 模型	MDLM
注意力机制	因果注意力	双向注意力
输入	干净文本 $\mathbf{x}$	噪声序列 $\mathbf{z}_t$ （随机掩码后）
额外采样	无	需采样时间步 t
损失函数	交叉熵	加权交叉熵（权重 $\frac{\alpha'_t}{1 - \alpha_t}$ ）
计算位置	所有位置	仅 mask 位置

训练简洁性

如果你知道如何训练一个 AR 模型，你基本上也知道如何训练一个 MDLM。唯一的新增操作是：（1）随机采样一个时间步 t ；（2）根据 t 对输入进行随机掩码；（3）在损失中加入权重项。这种简洁性是 MDLM 被广泛采用的重要原因。

3.3 采样过程详解

MDLM 的完整采样过程（祖先采样, Ancestral Sampling）：

1. **初始化**: $\mathbf{z}_T = [\text{M}, \text{M}, \dots, \text{M}]$ (全 mask 序列)
2. **单步去噪** (从 $\mathbf{z}_t$ 到 $\mathbf{z}_s$):
 - (a) 将 $\mathbf{z}_t$ 送入双向 Transformer
 - (b) 模型对所有 mask 位置预测词的分布
 - (c) 从分布中采样, 填充所有 mask 位置
 - (d) 根据去掩码概率 $\frac{\alpha_s - \alpha_t}{1 - \alpha_t}$ 决定保留哪些预测
 - (e) 未保留的预测被重新 mask
 - (f) 已揭示的 token 保持不变
3. 重复步骤 2 共 T 步, 直到 $\mathbf{z}_0$ 完全揭示

这不是启发式方法

这个采样过程不是凭直觉设计的启发式方法, 而是严格从数学推导中得出的祖先采样器 (Ancestral Sampler)。数学推导要求: (1) 已揭示的 token 必须保持不变; (2) 只有 mask 位置的预测才会被重新 mask。

3.4 本章小结

MDLM 的训练损失是一个加权交叉熵, 权重由去掩码概率决定。连续时间极限是 MDLM 相比前作的核心简化。训练流程与 AR 模型高度相似, 降低了实现门槛。

4 实验结果与应用

4.1 语言建模性能

4.1.1 LM1B 数据集

在 LM1B 数据集（上下文长度 128）上，MDLM 显著优于所有先前的离散扩散模型基线（包括最接近的基线 SEDD），并**接近了 AR 模型的困惑度**。

为什么困惑度重要

困惑度 (Perplexity) 是语言建模中最核心的评估指标。经验表明，如果一个模型在训练/验证集上取得了好的困惑度分数，它在下游任务（如编程、推理）上的表现也会更好。因此，困惑度是衡量语言模型质量的可靠代理指标。

当上下文长度增加到 1024 时，MDLM 仍然优于 SEDD，但与 AR 模型之间存在一定差距。不过，考虑到早期的扩散语言模型与 AR 基线差距巨大，这一结果已经非常令人鼓舞。

4.1.2 零样本迁移能力

将模型在 OpenWebText 上训练后，在 PTB、arXiv、PubMed 等 7 个未见过的数据集上评估零样本似然：

- MDLM 在所有数据集上一致优于 SEDD
- 在 7 个数据集集中的 3 个上，MDLM 的表现**优于 AR 基线**

4.2 大规模验证：LaDa (8B MDLM)

LaDa (Large Language Diffusion Models) 是 MDLM 在 80 亿参数规模上的验证。它本质上就是一个 8B 的 MDLM 模型。

MDLM 在大规模上的表现

- **多任务语言理解 (MMLU)**：MDLM 在整个训练过程中**一致超过 AR 基线**
- **推理任务**：MDLM 的性能扩展曲线呈**指数增长**，而 AR 基线仅为线性增长
- **常识问答和编程**：MDLM 与 AR 基线**持平或略低**

尽管在困惑度上仍有差距，但在实际下游任务上，MDLM 在多个方面超越了 AR 模型。

4.3 超越语言：离散数据建模

左右偏置与数据结构

自然语言具有固有的从左到右的偏置——这是人类说话和写作的方式。因此 AR 模型在语言任务上有天然优势。但在蛋白质生成、药物发现、分子生成等领域，数据**不存在左右偏置**，这正是 MDLM 大放异彩的场景。

在分子生成任务中 (GenMol 论文), 基于 MDLM 的模型展示了一个显著优势: 通过调节采样温度, 可以在分子**质量**和**多样性**之间获得一条 Pareto 前沿。相比之下, AR 基线只能给出一个固定的质量-多样性点, 无法进行这种灵活的权衡。

4.4 可控生成

MDLM 天然适合可控生成。原因在于:

1. 在去噪过程中, 模型会填充所有 mask 位置, 产生对最终样本的**完整预测**
2. 由此可以获得关于最终样本的高层信息
3. 利用这些信息, 可以在采样过程中引导生成具有特定属性的样本

AR 模型难以进行可控采样

AR 模型在可控生成方面存在众所周知的困难。因为在逐词生成的过程中, 模型无法预见未来的全局结构, 因此很难在生成过程中施加全局约束。MDLM 的双向注意力机制和全局去噪预测使其在可控生成上具有本质优势。

4.5 本章小结

实验结果表明: (1) MDLM 在小规模上接近 AR 性能, 在大规模下游任务 (特别是推理) 上甚至超过 AR; (2) 在无左右偏置的离散数据 (如分子生成) 上, MDLM 表现尤为突出; (3) MDLM 的采样灵活性使其适合可控生成等应用场景。

5 MDLM 的局限性：KV 缓存问题

5.1 KV 缓存回顾

在 AR 模型中，**KV 缓存** (Key-Value Caching) 是推理加速的关键技术。

KV 缓存的工作原理

在 AR Transformer 中：

1. 因为使用因果注意力，每个 token 的激活值只依赖左侧上下文
2. 已生成 token 的激活值**永远不会改变**，因为未来的 token 不影响它们
3. 因此可以缓存 (freeze) 已计算的 Key 和 Value
4. 生成新 token 时只需对**单个 token** 做前向传播，而非整个上下文

这使得 AR 模型即使需要上千步解码，每一步的计算成本都很低。

5.2 为什么 MDLM 不支持 KV 缓存

MDLM 使用双向注意力，这意味着**每个 token 的激活值依赖所有其他 token**。

考虑以下场景：

1. 初始状态：[M, M, M, M]
2. 第一步去噪后：[M, B, M, D] (揭示了位置 2 和 4)
3. 第二步去噪后：[A, B, M, D, M, F] (揭示了位置 1 和 6)

在第二步中，虽然 *B* 和 *D* 没有改变，但由于新出现了 *A* 和 *F*，在双向注意力下 *B* 和 *D* 的激活值会发生变化。因此**无法冻结任何位置的激活值**。

MDLM 的推理瓶颈

尽管 MDLM 可以用远少于序列长度的步数生成文本（例如用 400-500 步生成 1000 词），但每一步都需要对**整个序列**做完整的前向传播。对于长序列，注意力的 $O(n^2)$ 复杂度使得每步的成本远高于 AR 模型（带 KV 缓存）的单步成本。因此在实际推理时间上，MDLM 可能反而比 AR 模型更慢。

5.3 本章小结

KV 缓存的缺失是 MDLM 最大的实际局限性。双向注意力虽然赋予了模型全局感知能力，但也导致了激活值随输入变化而全局更新，阻碍了 KV 缓存的使用。

6 SoLM: 融合 AR 与扩散的排序语言模型

6.1 核心思路: 排序 + 因果注意力

SoLM (Sorting-based Language Model, 亦称 Esoteric Language Model) 的核心贡献是为 MDLM 引入 KV 缓存支持。关键思路:

SoLM 的训练策略

将 MDLM 的双向注意力替换为**因果注意力**, 通过以下操作实现:

1. 对噪声序列 $\mathbf{z}_t$ 进行**排序/洗牌**
2. 将所有**干净 token** 放到**左侧** (作为已知上下文)
3. 将所有 **mask token** 放到**右侧** (作为待预测目标)
4. 对排序后的序列施加**因果注意力**

在这种排列下:

- 第一个 mask token 只依赖干净上下文
- 第二个 mask token 依赖干净上下文 + 前一个 mask token
- 以此类推...
- 没有任何 mask token 依赖右侧的”未来” mask token

6.2 推理过程与 KV 缓存

SoLM 的推理过程充分利用了因果注意力带来的 KV 缓存能力:

1. 祖先采样器决定要去噪哪些 mask 位置 (例如位置 3 和 2)
2. **只对这些 mask token 做前向传播**, 附带对应的位置编码
3. 已预测 token 的激活值被**冻结** (缓存)
4. 下一步去噪时, 只需处理新的 mask token, 利用缓存的 KV

SoLM 推理的计算节省

对比 MDLM 每步需要处理整个序列 (包括大量不携带信息的 mask token), SoLM 每步**只处理待去噪的 token**。这带来了两方面的节省:

1. 前向传播的序列长度大幅缩短
2. KV 缓存避免了重复计算已处理 token 的激活

这两个因素共同实现了**数量级的推理加速**。

6.3 AR 与扩散的插值

SoLM 的另一个重要贡献是它能够在纯 AR 模式和纯扩散模式之间平滑插值：

- **纯 AR 模式**：每步只去噪一个 token（从左到右），质量最高但速度最慢
- **纯扩散模式**：每步去噪多个 token（并行），速度最快但可能有质量损失
- **中间模式**：根据需求在速度和质量之间灵活权衡

灵活的速度-质量权衡

这种插值能力意味着用户可以根据具体应用场景的需求来选择最优的工作点。例如，在对延迟敏感的在线服务中可以选择更激进的并行生成，在对质量要求极高的场景中则采用接近 AR 的模式。

6.4 本章小结

SoLM 通过排序 + 因果注意力的巧妙设计，在保持扩散模型并行生成能力的同时实现了 KV 缓存支持。这使得推理效率实现了数量级的提升，真正使扩散语言模型具备了实用化的推理速度。

7 方法对比与性能分析

7.1 MDLM vs BD3LM vs SoLM

BD3LM (Block Diffusion Language Model) 是另一种支持 KV 缓存的扩散语言模型方法。其思路是将长序列分成固定大小的块 (例如每块 100 个 token)，在每个块内做扩散去噪，块间则使用自回归方式推进。

表 2: 三种扩散语言模型方法对比

特性	MDLM	BD3LM	SoLM
注意力类型	双向	块内双向 + 块间因果	因果 (排序后)
KV 缓存	不支持	部分支持 (仅跨块)	完全支持
并行生成	全局并行	块内并行	全局并行
推理速度	慢	中等	快
生成质量	高	中-高	高

BD3LM 在少步采样时的质量退化

BD3LM 在增加采样步数时能维持较好的质量，但当大幅减少采样步数 (即每步并行生成更多词) 时，质量会显著退化，甚至可能不如 MDLM。这是因为块内扩散的步数减少后，去噪效果不足。相比之下，SoLM 在少步采样时仍能保持较好的质量。

7.2 速度-质量权衡曲线

在困惑度 vs 采样时间的 Pareto 前沿上：

- **MDLM**：困惑度最优，但推理速度最慢 (曲线偏右下方)
- **BD3LM**：在高采样步数时快于 MDLM，但减少步数后质量急剧下降
- **SoLM**：Pareto 前沿**严格优于** MDLM 和 BD3LM，即在任意给定速度下质量更高，或在任意给定质量下速度更快

在 110M 参数、1024 token 上下文长度的基准测试中，SoLM 相比 MDLM 的采样时间实现了显著的减少。

7.3 本章小结

SoLM 在速度-质量权衡上严格优于 MDLM 和 BD3LM，这主要归功于其完全的 KV 缓存支持。BD3LM 的块式设计虽然也能部分支持 KV 缓存，但块内仍受限于双向注意力的瓶颈。

8 总结与延伸

8.1 核心要点回顾

1. **扩散语言模型**打破了 AR 模型从左到右的约束，能够并行生成文本
2. **MDLM** 通过简洁的连续时间 ELBO，使离散扩散的训练和 AR 模型一样简单，性能在大规模上可匹配甚至超越 AR
3. **KV 缓存**是 MDLM 实际部署的最大瓶颈——双向注意力导致激活值无法冻结
4. **SoLM** 通过排序 + 因果注意力的设计，在保持并行生成能力的同时支持 KV 缓存，实现了数量级的推理加速
5. 在**无左右偏置**的离散数据任务（蛋白质、分子生成）上，扩散语言模型的优势尤为显著

8.2 工业应用现状

截至本讲座录制时（2025 年秋季），MDLM 框架已经在工业界得到广泛应用：

- **ByteDance Seed Diffusion**：基于 MDLM 框架构建
- **Google Gemini Diffusion**：采用扩散方法进行文本生成
- **Inception Labs Mercury Coder**：面向代码生成的扩散模型
- **NVIDIA**：使用 MDLM 进行分子生成（药物发现领域）

8.3 拓展阅读

- Sahoo et al., “Simple and Effective Masked Diffusion Language Models” (MDLM), NeurIPS 2024
- Sahoo et al., “Esoteric Language Models” (SoLM), 2025
- “Diffusion Duality”: 蒸馏算法，10 步生成 1000 词
- LaDa: Large Language Diffusion Models (8B 规模 MDLM 验证)
- BD3LM: Block Diffusion Language Models
- GenMol: 基于 MDLM 的分子生成
- KAIST CS492D 课程主页: <https://mhsung.github.io/kaist-cs492d-fall-2025/>